

SIMATS ENGINEERING

ASSESSMENT TOOL 1: CODING CHALLENGE

Course Code: CSA51 | Course Name: Cryptography and Network Security

Student Name: T.Lakshmi Prasad Naidu

Register Number: 192324187

Course Outcome: CO4 (BL3, BL6)

Assessment Weightage: 40% (Total Marks: 25)

Code of Conduct Declaration:

I, T.Lakshmi Prasad Naidu (Reg No: 192324187), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: T.Lakshmi Prasad Naidu

Question 1: Secure Email Transmission using S/MIME Simulation

Problem Statement: Write a Python program to simulate Secure Email Transmission using S/MIME, including message encryption, digital signature generation, and verification. Analyze how confidentiality and authentication are ensured.

Python Implementation

```
import hashlib
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os

# 1. Key Generation for Sender (Alice) and Receiver (Bob)
sender_private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
sender_public_key = sender_private_key.public_key()

receiver_private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
receiver_public_key = receiver_private_key.public_key()

def sime_secure_send(message: str, s_priv, r_pub):
    msg_bytes = message.encode('utf-8')

    # Step A: Digital Signature (Sender signs SHA-256 hash of message)
    signature = s_priv.sign(
        msg_bytes,
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH),
        hashes.SHA256()
    )

    # Combine message + signature
```

```

payload = msg_bytes + b"::SIG::" + signature

# Step B: Message Encryption using Session Key (AES-256-CBC)
session_key = os.urandom(32)
iv = os.urandom(16)
pad_len = 16 - (len(payload) % 16)
padded_payload = payload + bytes([pad_len] * pad_len)

cipher = Cipher(algorithms.AES(session_key), modes.CBC(iv))
encryptor = cipher.encryptor()
ciphertext = encryptor.update(padded_payload) + encryptor.finalize()

# Step C: Asymmetrically encrypt session key with Receiver's Public Key
enc_session_key = r_pub.encrypt(
    session_key,
    padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()),
algorithm=hashes.SHA256(), label=None)
)

return iv, enc_session_key, ciphertext

def sime_secure_receive(iv, enc_session_key, ciphertext, r_priv, s_pub):
    # Step 1: Decrypt Session Key using Receiver's Private Key
    session_key = r_priv.decrypt(
        enc_session_key,
        padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()),
algorithm=hashes.SHA256(), label=None)
    )

    # Step 2: Decrypt Ciphertext using Session Key
    cipher = Cipher(algorithms.AES(session_key), modes.CBC(iv))
    decryptor = cipher.decryptor()
    padded_payload = decryptor.update(ciphertext) + decryptor.finalize()
    pad_len = padded_payload[-1]
    payload = padded_payload[:-pad_len]

    # Step 3: Separate Message and Signature
    msg_bytes, signature = payload.split(b"::SIG::")

    # Step 4: Verify Digital Signature using Sender's Public Key
    try:
        s_pub.verify(
            signature,
            msg_bytes,
            padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH),
            hashes.SHA256()
        )
        return msg_bytes.decode('utf-8'), True
    except Exception:
        return msg_bytes.decode('utf-8'), False

# Test Simulation

```

```

email_text = "Confidential: Project CSA51 Final Deliverable."
iv, enc_key, ciphertext = sime_secure_send(email_text, sender_private_key,
receiver_public_key)
recovered_msg, is_verified = sime_secure_receive(iv, enc_key, ciphertext,
receiver_private_key, sender_public_key)

print(f"Decrypted Message: {recovered_msg}")
print(f"Signature Validated (Authentication & Integrity): {is_verified}")

```

Security Analysis: Confidentiality and Authentication

- **Confidentiality:** Achieved via hybrid encryption. Symmetric key (AES-256) encrypts the bulk payload efficiently, while asymmetric RSA-OAEP encrypts the session key using recipient's public key, preventing eavesdropping.
- **Authentication & Non-Repudiation:** Achieved via RSA-PSS digital signatures over SHA-256 message hashes. Only the owner of the sender's private key could generate the valid signature, verifying sender authenticity and ensuring tamper-evident integrity.

Question 2: PGP-Based File Encryption and Decryption

Problem Statement: Develop a Python application to implement PGP-based file encryption and decryption, including key generation, key distribution, and message authentication. Evaluate how PGP provides both security and efficiency.

Python Implementation

```

import os
import zlib
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

class PGFileSecurity:
    @staticmethod
    def generate_keypair():
        private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
        return private_key, private_key.public_key()

    @staticmethod
    def encrypt_and_sign_file(input_filepath, output_filepath, sender_priv_key,
receiver_pub_key):
        with open(input_filepath, 'rb') as f:
            data = f.read()

        # Step 1: Sign Data
        signature = sender_priv_key.sign(
            data,
            padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH),
            hashes.SHA256())

```

```

    )

    # Step 2: Compress (Data + Signature) using DEFLATE
    bundle = len(signature).to_bytes(4, 'big') + signature + data
    compressed_bundle = zlib.compress(bundle)

    # Step 3: Symmetric Encryption (AES-256-CFB)
    session_key = os.urandom(32)
    iv = os.urandom(16)
    cipher = Cipher(algorithms.AES(session_key), modes.CFB(iv))
    encryptor = cipher.encryptor()
    encrypted_data = encryptor.update(compressed_bundle) + encryptor.finalize()

    # Step 4: Encrypt Session Key
    enc_session_key = receiver_pub_key.encrypt(
        session_key,
        padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()),
algorithm=hashes.SHA256(), label=None)
    )

    with open(output_filepath, 'wb') as f:
        f.write(len(enc_session_key).to_bytes(4, 'big') + enc_session_key + iv +
encrypted_data)

    @staticmethod
    def decrypt_and_verify_file(encrypted_filepath, output_filepath, receiver_priv_key,
sender_pub_key):
        with open(encrypted_filepath, 'rb') as f:
            raw = f.read()

        key_len = int.from_bytes(raw[:4], 'big')
        enc_session_key = raw[4:4 + key_len]
        iv = raw[4 + key_len: 4 + key_len + 16]
        encrypted_data = raw[4 + key_len + 16:]

        # Step 1: Decrypt Session Key
        session_key = receiver_priv_key.decrypt(
            enc_session_key,
            padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()),
algorithm=hashes.SHA256(), label=None)
        )

        # Step 2: Decrypt Ciphertext
        cipher = Cipher(algorithms.AES(session_key), modes.CFB(iv))
        decryptor = cipher.decryptor()
        compressed_bundle = decryptor.update(encrypted_data) + decryptor.finalize()

        # Step 3: Decompress
        bundle = zlib.decompress(compressed_bundle)
        sig_len = int.from_bytes(bundle[:4], 'big')
        signature = bundle[4:4 + sig_len]
        original_data = bundle[4 + sig_len:]

```

```

# Step 4: Verify Signature
sender_pub_key.verify(
    signature,
    original_data,
    padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH),
    hashes.SHA256()
)

with open(output_filepath, 'wb') as f:
    f.write(original_data)
return True

# Example Usage
alice_priv, alice_pub = PGPSecurity.generate_keypair()
bob_priv, bob_pub = PGPSecurity.generate_keypair()

with open("test_file.txt", "w") as f:
    f.write("PGP ensures confidentiality, authenticity, and compression efficiency.")

PGPSecurity.encrypt_and_sign_file("test_file.txt", "encrypted.pgp", alice_priv,
bob_pub)
is_valid = PGPSecurity.decrypt_and_verify_file("encrypted.pgp", "restored.txt",
bob_priv, alice_pub)
print(f"PGP File Decryption & Signature Verification Success: {is_valid}")

```

Evaluation: Security and Efficiency in PGP

- **Efficiency:** By compressing the file and signature bundle with DEFLATE prior to symmetric encryption, PGP optimizes disk and bandwidth utilization. Furthermore, compression flattens frequency distributions, resisting cryptanalysis.
- **Security & Key Management:** PGP uses a decentralized 'Web of Trust' model for public key verification. By computing digital signatures before compression and encryption, it ensures non-repudiation and unalterable verification.

Question 3: IPSec Tunnel Mode Simulation

Problem Statement: Implement a Python program to simulate IPSec Tunnel Mode, encrypting an entire IP packet using symmetric encryption. Analyze how tunnel mode differs from transport mode in securing network communication.

Python Implementation

```

import os
import json
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

class IPSecTunnelSimulator:
    def __init__(self, pre_shared_key: bytes):
        self.key = pre_shared_key

```

```

def encapsulate_and_encrypt(self, original_ip_packet: dict, tunnel_src: str,
tunnel_dst: str):
    raw_packet = json.dumps(original_ip_packet).encode('utf-8')

    # Pad payload to 16-byte boundary
    pad_len = 16 - (len(raw_packet) % 16)
    padded_payload = raw_packet + bytes([pad_len] * pad_len)

    iv = os.urandom(16)
    cipher = Cipher(algorithms.AES(self.key), modes.CBC(iv))
    encryptor = cipher.encryptor()
    encrypted_esp_payload = encryptor.update(padded_payload) + encryptor.finalize()

    # Build New Outer IP Header (Tunnel Header) + ESP encapsulation
    tunnel_packet = {
        "outer_header": {
            "version": 4,
            "src_gateway": tunnel_src,
            "dst_gateway": tunnel_dst,
            "protocol": "ESP (50)"
        },
        "esp_header": {
            "spi": 0x1001,
            "seq_no": 1,
            "iv": iv.hex()
        },
        "esp_payload": encrypted_esp_payload.hex()
    }
    return tunnel_packet

def decapsulate_and_decrypt(self, tunnel_packet: dict):
    iv = bytes.fromhex(tunnel_packet["esp_header"]["iv"])
    encrypted_payload = bytes.fromhex(tunnel_packet["esp_payload"])

    cipher = Cipher(algorithms.AES(self.key), modes.CBC(iv))
    decryptor = cipher.decryptor()
    padded_payload = decryptor.update(encrypted_payload) + decryptor.finalize()

    pad_len = padded_payload[-1]
    raw_packet = padded_payload[:-pad_len]
    original_packet = json.loads(raw_packet.decode('utf-8'))
    return original_packet

# Demonstration
psk = os.urandom(32) # AES-256 key
gateway_a = IPSecTunnelSimulator(psk)
gateway_b = IPSecTunnelSimulator(psk)

original_inner_packet = {
    "src_ip": "192.168.1.50",
    "dst_ip": "10.0.0.99",
    "protocol": "TCP",
    "data": "GET /classified-data HTTP/1.1"

```

```

}

tunnel_pkt = gateway_a.encapsulate_and_encrypt(original_inner_packet, "203.0.113.1",
"198.51.100.2")
print("Encapsulated Tunnel Packet:")
print(json.dumps(tunnel_pkt, indent=2))

decrypted_inner_packet = gateway_b.decapsulate_and_decrypt(tunnel_pkt)
print("\nDecapsulated Inner Packet:")
print(json.dumps(decrypted_inner_packet, indent=2))

```

Comparison: IPSec Tunnel Mode vs. Transport Mode

Parameter	Tunnel Mode	Transport Mode
Encapsulation Scope	Encrypts entire original IP packet (Header + Payload) and prepends a new Outer IP Header.	Encrypts only the transport-layer payload/data; original IP Header remains unencrypted.
Traffic Analysis Protection	High: Intermediate nodes only see gateway IPs; original internal host IPs are concealed.	Low: Source and destination host IP addresses remain in the clear across all hops.
Primary Use Case	Gateway-to-Gateway (VPN site-to-site) or Host-to-Gateway remote access.	End-to-End direct Host-to-Host communication within the same network.

Question 4: SSL/TLS Handshake Simulation

Problem Statement: Write a Python script to implement SSL/TLS handshake simulation, including certificate exchange, session key generation, and secure data transfer. Evaluate how the handshake ensures secure web communication.

Python Implementation

```

import os
import hashlib
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

class TLSHandshakeSimulation:
    def __init__(self):
        self.server_priv_key = rsa.generate_private_key(public_exponent=65537,
key_size=2048)
        self.server_pub_key = self.server_priv_key.public_key()
        self.server_certificate = {"issuer": "Global-CA", "subject": "bank.com",
"pub_key": self.server_pub_key}

    def simulate_handshake_and_transfer(self):
        # 1. ClientHello
        client_random = os.urandom(32)

```

```

print("[Step 1] Client sends ClientHello (ClientRandom, CipherSuites)")

# 2. ServerHello + Certificate
server_random = os.urandom(32)
cert = self.server_certificate
print("[Step 2] Server sends ServerHello, Selected Cipher, and Certificate")

# 3. Client Key Exchange (Pre-Master Secret generation & asymmetric encryption)
pre_master_secret = os.urandom(48)
enc_pre_master = cert["pub_key"].encrypt(
    pre_master_secret,
    padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()),
algorithm=hashes.SHA256(), label=None)
)
print("[Step 3] Client encrypts Pre-Master Secret with Server's Public Key")

# 4. Server Decrypts Pre-Master Secret
dec_pre_master = self.server_priv_key.decrypt(
    enc_pre_master,
    padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()),
algorithm=hashes.SHA256(), label=None)
)

# 5. Master Secret & Session Key Derivation (PRF simulation)
master_secret = hashlib.sha256(pre_master_secret + b"master secret" +
client_random + server_random).digest()
session_key = hashlib.sha256(master_secret + b"key expansion" + server_random +
client_random).digest()[:16]
print(f"[Step 4] Both parties derive symmetric Session Key:
{session_key.hex()[:16]}...")

# 6. Secure Data Transfer via Session Key
iv = os.urandom(16)
http_data = b"POST /login HTTP/1.1\r\nUser: admin\r\nPass: 12345"
pad_len = 16 - (len(http_data) % 16)
padded_http = http_data + bytes([pad_len] * pad_len)

cipher = Cipher(algorithms.AES(session_key), modes.CBC(iv))
enc_http = cipher.encryptor().update(padded_http) + cipher.encryptor().finalize()
print(f"[Step 5] Encrypted Data Transferred over TLS tunnel:
{enc_http.hex()[:32]}...")

tls = TLSHandshakeSimulation()
tls.simulate_handshake_and_transfer()

```

Evaluation: How Handshake Secures Web Communication

- **Server Authentication:** Prevents Man-in-the-Middle (MitM) attacks by verifying that the web server owns the identity bound in the trusted CA certificate.

- **Ephemeral Symmetric Key Derivation:** Leverages PRF mixing (ClientRandom, ServerRandom, and Pre-Master Secret) to produce session keys without transmitting the master key across public networks.
- **Session Integrity & Confidentiality:** All subsequent application layer payload (HTTPS) is encrypted symmetrically and integrity-verified with MACs, defending against wiretapping and packet manipulation.

Question 5: Email Phishing Detection System

Problem Statement: Build a Python-based tool to perform Email Phishing Detection, using feature extraction (URL patterns, headers, keywords) and classification techniques. Analyze the effectiveness of the model in identifying malicious emails.

Python Implementation

```
import re
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.base import BaseEstimator, TransformerMixin
import numpy as np

emails_corpus = [
    ("URGENT: Your bank account is locked! Click http://192.168.1.1/login to verify your password immediately.", 1),
    ("Security Alert: Suspicious login detected. Update payment info now at http://paypal.security-update.com/signin", 1),
    ("Claim your free $1,000 gift card now by entering credentials at http://free-rewards.xyz/claim", 1),
    ("Meeting agenda for next Monday's security engineering review attached.", 0),
    ("The draft for the Cryptography and Network Security course assignment has been uploaded to portal.", 0),
    ("Let's grab lunch tomorrow to discuss the final project timeline.", 0)
]

class StructuralFeatureExtractor(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        return self

    def transform(self, X):
        features = []
        for text in X:
            has_ip_url = 1 if re.search(r'https?:\/\/\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}', text) else 0
            has_suspicious_domain = 1 if re.search(r'\.(xyz|top|security-update|free-)\b', text) else 0
            num_urls = len(re.findall(r'https?:\/\/\S+', text))
            urgent_keywords = len(re.findall(r'\b(urgent|verify|password|locked|alert|claim|free|update)\b', text, re.IGNORECASE))
            features.append([has_ip_url, has_suspicious_domain, num_urls, urgent_keywords])
```

```

        return np.array(features)

X_train = [t[0] for t in emails_corpus]
y_train = [t[1] for t in emails_corpus]

# Feature extraction & classification pipeline
vectorizer = TfidfVectorizer()
X_tfidf = vectorizer.fit_transform(X_train).toarray()
struct_extractor = StructuralFeatureExtractor()
X_struct = struct_extractor.transform(X_train)
X_combined = np.hstack((X_tfidf, X_struct))

clf = MultinomialNB()
clf.fit(X_combined, y_train)

def detect_phishing(email_text: str):
    feat_tfidf = vectorizer.transform([email_text]).toarray()
    feat_struct = struct_extractor.transform([email_text])
    feat_combined = np.hstack((feat_tfidf, feat_struct))

    pred = clf.predict(feat_combined)[0]
    prob = clf.predict_proba(feat_combined)[0][1]
    return "Phishing" if pred == 1 else "Legitimate", prob

# Test Samples
sample_1 = "URGENT: Verify your account immediately at http://10.0.0.1/verify"
sample_2 = "Can you send me the CSA51 lecture notes?"

label1, prob1 = detect_phishing(sample_1)
label2, prob2 = detect_phishing(sample_2)

print(f"Email 1: {label1} (Phishing Probability: {prob1:.2f})")
print(f"Email 2: {label2} (Phishing Probability: {prob2:.2f})")

```

Effectiveness Analysis of Phishing Detection Models

- **Multi-Modal Feature Extraction:** Coupling lexical NLP features (TF-IDF keyword n-grams) with heuristic structural rules (IP-in-URL, deceptive domain TLDs, urgent sentiment triggers) captures both social engineering tactics and technical evasion patterns.
- **Algorithmic Performance:** Multinomial Naive Bayes performs exceptionally well with high-dimensional sparse text representations, delivering near real-time edge filtering with minimal memory footprint.
- **Robustness & Defense:** To counter adversarial evasion (such as homograph attacks or image-based spam), the classification layer is combined with DKIM/SPF/DMARC authentication protocols and optical character recognition (OCR).